

// Jivops

Guía completa — 30 lecciones

Arquitectura · Gestión a escala · Seguridad y RBAC · Multi-cluster ·
Despliegue progresivo · Operación avanzada

Contenido

Módulo 1: Fundamentos y arquitectura

- Lección 01 — Arquitectura de ArgoCD — componentes
- Lección 02 — Instalación y configuración inicial
- Lección 03 — Application CRD — fundamentos
- Lección 04 — Sync policies — manual vs automated
- Lección 05 — Sync waves y hooks

Módulo 2: Gestión de aplicaciones a escala

- Lección 06 — App of Apps pattern
- Lección 07 — ApplicationSets — generadores
- Lección 08 — Proyectos de ArgoCD (AppProject)
- Lección 09 — Gestión de Helm charts con ArgoCD
- Lección 10 — Kustomize con ArgoCD

Módulo 3: Seguridad y RBAC

- Lección 11 — RBAC en ArgoCD
- Lección 12 — SSO con Dex/OIDC
- Lección 13 — Gestión de secretos (Sealed Secrets, External Secrets)
- Lección 14 — Repository credentials y acceso a Git privado
- Lección 15 — Auditoría y logging de cambios

Módulo 4: Multi-cluster y escalado

- Lección 16 — Gestión multi-cluster con ArgoCD
- Lección 17 — Cluster de control central vs distribuido
- Lección 18 — Alta disponibilidad de ArgoCD
- Lección 19 — Escalado del repo server
- Lección 20 — Namespace-scoped Argo CD instances

Módulo 5: Despliegue progresivo y observabilidad

- Lección 21 — Argo Rollouts — Canary deployments
- Lección 22 — Argo Rollouts — Blue-Green deployments
- Lección 23 — Analysis templates y métricas automáticas
- Lección 24 — Notificaciones (Argo CD Notifications)
- Lección 25 — Dashboards y monitoreo con Prometheus

Módulo 6: Operación avanzada

Lección 26 — Drift detection y self-healing

Lección 27 — Backup y disaster recovery de ArgoCD

Lección 28 — Migración y actualización de ArgoCD

Lección 29 — Integración con Vault/External Secrets Operator

Lección 30 — Proyecto final: ArgoCD completo listo para producción

Módulo 1: Fundamentos y arquitectura

Lección 01 — Arquitectura de ArgoCD — componentes

El Application Controller compara continuamente el estado deseado en Git con el real del cluster. Separar componentes permite escalar cada uno según su carga específica.

Lección 02 — Instalación y configuración inicial

Cambiar la contraseña por defecto del admin es crítico: dejarla sin cambiar daría control total sobre todas las aplicaciones a cualquiera que la conozca.

```
argocd admin initial-password -n argocd
```

Lección 03 — Application CRD — fundamentos

El recurso Application define la fuente en Git y el destino. Al ser un YAML declarativo, se versiona en Git siguiendo el propio principio de GitOps.

Lección 04 — Sync policies — manual vs automated

Con automated, ArgoCD aplica cambios detectados sin intervención; manual requiere confirmación humana, útil en entornos de producción muy críticos.

```
syncPolicy: automated: prune: true
```

Lección 05 — Sync waves y hooks

Las sync waves controlan el orden de aplicación de recursos, garantizando por ejemplo que una base de datos esté lista antes que la app que depende de ella.

```
argocd.argoproj.io/sync-wave: "1"
```

Módulo 2: Gestión de aplicaciones a escala

Lección 06 — App of Apps pattern

Una Application raíz orquesta múltiples Applications hijas desde Git. Añadir una nueva app es solo agregar un manifiesto, sin crear cada una manualmente.

Lección 07 — ApplicationSets — generadores

Un generador de tipo Cluster crea automáticamente Applications para todos los clusters registrados, útil para desplegar la misma app en muchos clusters sin duplicar manifiestos.

Lección 08 — Proyectos de ArgoCD (AppProject)

Un AppProject restringe qué repos, clusters y namespaces puede usar un conjunto de Applications, evitando errores o accesos entre equipos distintos.

Lección 09 — Gestión de Helm charts con ArgoCD

Usar distintos valueFiles (dev, staging, prod) con el mismo chart base evita duplicar todo el chart completo para cada entorno.

```
helm: valueFiles: - values-prod.yaml
```

Lección 10 — Kustomize con ArgoCD

Un overlay de Kustomize aplica parches específicos sobre una base común; un cambio en la base se propaga a todos los entornos sin duplicar manifiestos completos.

Módulo 3: Seguridad y RBAC

Lección 11 — RBAC en ArgoCD

Dar permiso de 'sync' pero no de 'delete' a desarrolladores aplica menor privilegio, permitiendo la acción común pero restringiendo acciones destructivas.

Lección 12 — SSO con Dex/OIDC

Dex actúa como conector OIDC entre ArgoCD y proveedores de identidad externos, facilitando revocar acceso de un empleado desde un solo lugar.

Lección 13 — Gestión de secretos (Sealed Secrets, External Secrets)

Un Kubernetes Secret solo está codificado en base64, no cifrado. Sealed Secrets cifra el valor antes de subirlo a Git, y solo el cluster destino puede descifrarlo.

Lección 14 — Repository credentials y acceso a Git privado

Usar una clave SSH con permisos de solo lectura para ArgoCD es más seguro que credenciales de escritura completa, ya que ArgoCD solo necesita leer el repo.

```
argocd repo add --ssh-private-key-path ~/.ssh/id_rsa
```

Lección 15 — Auditoría y logging de cambios

El historial de sync muestra exactamente qué commit se aplicó y cuándo, clave para investigar qué cambio causó un problema tras un despliegue.

Módulo 4: Multi-cluster y escalado

Lección 16 — Gestión multi-cluster con ArgoCD

Registrar múltiples clusters en una sola instalación da visibilidad y control unificado desde un solo panel, en vez de instalaciones separadas por cluster.

```
argocd cluster add mi-contexto
```

Lección 17 — Cluster de control central vs distribuido

Un hub central se convierte en punto único de fallo para todos los despliegues gestionados; instancias distribuidas aíslan el impacto por cluster.

Lección 18 — Alta disponibilidad de ArgoCD

Replicar el repo server y API server distribuye la carga de peticiones, evitando que una sincronización pesada bloquee al resto de aplicaciones.

```
kubectl scale deployment argocd-repo-server --replicas=3
```

Lección 19 — Escalado del repo server

Ajustar la caché de manifiestos renderizados evita reprocesar el mismo manifiesto en cada sync si no ha cambiado desde la última vez.

Lección 20 — Namespace-scoped Argo CD instances

Instancias namespace-scoped dan aislamiento real entre equipos en entornos multi-tenant, evitando que uno vea o afecte las aplicaciones de otro.

Módulo 5: Despliegue progresivo y observabilidad

Lección 21 — Argo Rollouts — Canary deployments

Un Canary dirige gradualmente un porcentaje creciente de tráfico a la nueva versión; si algo falla, solo ese porcentaje pequeño se ve afectado.

```
strategy: canary: steps: - setWeight: 20
```

Lección 22 — Argo Rollouts — Blue-Green deployments

Blue-Green mantiene dos versiones completas en paralelo, permitiendo un rollback casi instantáneo ya que la versión anterior sigue corriendo intacta.

```
strategy: blueGreen: activeService: miapp-active
```

Lección 23 — Analysis templates y métricas automáticas

Un AnalysisTemplate evalúa métricas reales (como tasa de errores) durante el rollout, decidiendo automáticamente continuar, pausar o revertir.

Lección 24 — Notificaciones (Argo CD Notifications)

Un trigger para sync fallido envía una alerta automática, acelerando la respuesta del equipo frente a descubrirlo manualmente después.

```
trigger: on-sync-failed
```

Lección 25 — Dashboards y monitoreo con Prometheus

Un dashboard con el estado de sync de todas las Applications da una vista general de un vistazo, en vez de revisar cada una individualmente.

Módulo 6: Operación avanzada

Lección 26 — Drift detection y self-healing

Con self-healing activado, ArgoCD revierte automáticamente un cambio manual hecho directamente en el cluster, protegiendo a Git como única fuente de verdad.

```
syncPolicy: automated: selfHeal: true
```

Lección 27 — Backup y disaster recovery de ArgoCD

Como Git ya es la fuente de verdad del estado deseado, gran parte de la recuperación es reinstalar ArgoCD y apuntarlo de vuelta a los mismos repos.

Lección 28 — Migración y actualización de ArgoCD

Probar la actualización en staging detecta cambios incompatibles en CRDs o comportamiento de sync antes de arriesgar aplicaciones críticas de producción.

Lección 29 — Integración con Vault/External Secrets Operator

Git solo contiene la referencia al secreto, no el valor real, manteniendo el flujo GitOps completo sin exponer datos sensibles en texto plano.

Lección 30 — Proyecto final: ArgoCD completo listo para producción

Se integra todo lo anterior: RBAC, App of Apps/ApplicationSets, Argo Rollouts, self-healing, alta disponibilidad y backup de configuración — el nivel esperado de un ArgoCD listo para producción real.